



UNIVERSIDAD TÉCNICA DE AMBATO

Facultad de Ingeniería en Sistemas, Electrónica e Industrial



Base de Datos Taller Practico

CARRERA:
Software

Nivel y Paralelo:
Cuarto "A"

Estudiante:

Fiallos Yanza Jonathan Josué

Asignatura:
Base De Datos

Docente:
Ing. Edwin Buenaño Valencia



Ejercicios Propuestos

Base de datos Banco

Accederemos a la base de datos previamente creada en SQL Plus para insertar más datos en las tablas existentes, permitiendo realizar consultas de manera más eficiente:

```
CONNECT SYSTEM
CREATE USER BANCO IDENTIFIED BY B123;
GRANT CONNECT, RESOURCE, UNLIMITED TABLESPACE TO BANCO;
CONNECT BANCO
```

```
CLIENTES=(ID_CLI, NOM_CLI, APE_CLI, DIR_CLI)
```

```
CUENTAS=(NUM_CUE, TIP_CUE, SAL_CUE, ANIO_APE, ID_CLI_PER)
```

```
CAJEROS_AUT=(ID_CAJ, UBI_CAJ, FEC_CAL_CAJ)
```

```
TRANSFERENCIAS=(NUM_TRA, FEC_HOR_TRA, VALOR_TRA, CUE_ORI, CUE_DES)
```

Creación de las tablas:

```
CREATE TABLE CLIENTES (
  ID_CLI VARCHAR(10) PRIMARY KEY,
  NOM_CLI VARCHAR(20) NOT NULL,
  APE_CLI VARCHAR(20) NOT NULL,
  DIR_CLI VARCHAR(50) NOT NULL
);
```

```
CREATE TABLE CUENTAS (
  NUM_CUE VARCHAR(15) PRIMARY KEY,
  TIP_CUE VARCHAR(10) NOT NULL,
  SAL_CUE NUMBER NOT NULL,
  ANIO_APE INT NOT NULL,
  ID_CLI_PER VARCHAR(10) NOT NULL,
  FOREIGN KEY (ID_CLI_PER) REFERENCES CLIENTES(ID_CLI)
);
```

```
CREATE TABLE CAJEROS_AUT (
  ID_CAJ VARCHAR(10) PRIMARY KEY,
  UBI_CAJ VARCHAR(50) NOT NULL,
  FEC_CAL_CAJ DATE NOT NULL
);
```

```
CREATE TABLE TRANSFERENCIAS (
  NUM_TRA NUMBER PRIMARY KEY,
  FEC_HOR_TRA DATE NOT NULL,
  VALOR_TRA NUMBER NOT NULL,
  CUE_ORI VARCHAR(15) NOT NULL,
```



CUE_DES VARCHAR(15) NOT NULL,
FOREIGN KEY (CUE_ORI) REFERENCES CUENTAS(NUM_CUE),
FOREIGN KEY (CUE_DES) REFERENCES CUENTAS(NUM_CUE)
);

Insertar datos :

CLIENTES

INSERT INTO CLIENTES (ID_CLI, NOM_CLI, APE_CLI, DIR_CLI)
VALUES ('C001', 'Ana', 'Martínez', 'Av. Central 123');

INSERT INTO CLIENTES (ID_CLI, NOM_CLI, APE_CLI, DIR_CLI)
VALUES ('C002', 'Luis', 'García', 'Calle Norte 45');

INSERT INTO CLIENTES (ID_CLI, NOM_CLI, APE_CLI, DIR_CLI)
VALUES ('C003', 'Sofía', 'Pérez', 'Av. Sur 89');

CUENTAS

INSERT INTO CUENTAS (NUM_CUE, TIP_CUE, SAL_CUE, ANIO_APE, ID_CLI_PER)
VALUES ('ACC1001', 'Ahorros', 1500.50, 2022, 'C001');

INSERT INTO CUENTAS (NUM_CUE, TIP_CUE, SAL_CUE, ANIO_APE, ID_CLI_PER)
VALUES ('ACC1002', 'Corriente', 2300.00, 2021, 'C002');

INSERT INTO CUENTAS (NUM_CUE, TIP_CUE, SAL_CUE, ANIO_APE, ID_CLI_PER)
VALUES ('ACC1003', 'Ahorros', 900.75, 2023, 'C003');

CAJEROS_AUT

INSERT INTO CAJEROS_AUT (ID_CAJ, UBI_CAJ, FEC_CAL_CAJ)
VALUES ('CJ01', 'Centro Comercial Plaza', '06/04/2025');

INSERT INTO CAJEROS_AUT (ID_CAJ, UBI_CAJ, FEC_CAL_CAJ)
VALUES ('CJ02', 'Estación Norte', '09/06/2025');

INSERT INTO CAJEROS_AUT (ID_CAJ, UBI_CAJ, FEC_CAL_CAJ)
VALUES ('CJ03', 'Terminal Terrestre', '02/03/2025');



PROCEDURE DEPOSITAR(NUM_CUE, VALOR_DEP, ID_CAJ_DEP)

```
SQL> CREATE OR REPLACE PROCEDURE DEPOSITAR (  
2   P_NUM_CUE   IN VARCHAR2,  
3   P_VALOR_DEP IN NUMBER,  
4   P_ID_CAJ_DEP IN VARCHAR2  
5 ) AS  
6 BEGIN  
7   -- Intentar actualizar la cuenta  
8   UPDATE CUENTAS  
9   SET SAL_CUE = SAL_CUE + P_VALOR_DEP  
10  WHERE NUM_CUE = P_NUM_CUE;  
11  
12  -- Si no se afectó ninguna fila, la cuenta no existe  
13  IF SQL%ROWCOUNT = 0 THEN  
14    RAISE_APPLICATION_ERROR(-20001, 'La cuenta no existe.');15  END IF;  
16  
17  COMMIT;  
18 END;  
19 /
```

Procedimiento creado.

Prueba del Procedure:

```
SQL> SELECT * FROM CUENTAS;
```

NUM_CUE	TIP_CUE	SAL_CUE	ANIO_APE	ID_CLI_PER
ACC1001	Ahorros	1500,5	2022	C001
ACC1002	Corriente	2300	2021	C002
ACC1003	Ahorros	900,75	2023	C003

```
SQL> EXECUTE DEPOSITAR('ACC1001', 500, 'CJ01');
```

Procedimiento PL/SQL terminado correctamente.

```
SQL> SELECT * FROM CUENTAS;
```

NUM_CUE	TIP_CUE	SAL_CUE	ANIO_APE	ID_CLI_PER
ACC1001	Ahorros	2000,5	2022	C001
ACC1002	Corriente	2300	2021	C002
ACC1003	Ahorros	900,75	2023	C003

Justificación:

Atomicidad: Se actualiza el saldo solo si la cuenta existe. Si falla, no se realiza ninguna acción parcial.

Consistencia: El saldo solo se incrementa si la cuenta existe, respetando reglas de integridad.

Aislamiento: Si múltiples depósitos ocurren, cada uno se ejecuta de forma aislada.

Durabilidad: El COMMIT asegura que el cambio se guarde permanentemente.



PROCEDURE RETIRAR(NUM CUE, VALOR RET, ID CAJ)

```
SQL> CREATE OR REPLACE PROCEDURE RETIRAR (  
  2   P_NUM_CUE IN VARCHAR,  
  3   P_VALOR_RET IN NUMBER,  
  4   P_ID_CAJ IN VARCHAR  
  5 ) AS  
  6   V_SALDO NUMBER;  
  7 BEGIN  
  8   -- Obtener saldo actual  
  9   SELECT SAL_CUE INTO V_SALDO  
10   FROM CUENTAS  
11   WHERE NUM_CUE = P_NUM_CUE;  
12  
13   IF V_SALDO < P_VALOR_RET THEN  
14     RAISE_APPLICATION_ERROR(-20002, 'Fondos insuficientes.');
```

```
15   END IF;  
16  
17   -- Descontar valor  
18   UPDATE CUENTAS  
19   SET SAL_CUE = SAL_CUE - P_VALOR_RET  
20   WHERE NUM_CUE = P_NUM_CUE;  
21  
22   COMMIT;  
23 END;  
24 /
```

Procedimiento creado.

Verificación del funcionamiento:

```
SQL> SELECT * FROM CUENTAS;
```

NUM_CUE	TIP_CUE	SAL_CUE	ANIO_APE	ID_CLI_PER
ACC1001	Ahorros	2000,5	2022	C001
ACC1002	Corriente	2300	2021	C002
ACC1003	Ahorros	900,75	2023	C003

```
SQL> EXECUTE RETIRAR('ACC1001', 3000, 'CJ03');  
BEGIN RETIRAR('ACC1001', 3000, 'CJ03'); END;
```

```
*  
ERROR en línea 1:  
ORA-20002: Fondos insuficientes.  
ORA-06512: en "BANCO.RETIRAR", línea 14  
ORA-06512: en línea 1
```

```
SQL> EXECUTE RETIRAR('ACC1001', 300, 'CJ03');
```

Procedimiento PL/SQL terminado correctamente.

```
SQL> SELECT * FROM CUENTAS;
```

NUM_CUE	TIP_CUE	SAL_CUE	ANIO_APE	ID_CLI_PER
ACC1001	Ahorros	1700,5	2022	C001
ACC1002	Corriente	2300	2021	C002
ACC1003	Ahorros	900,75	2023	C003



Justificación:

Atomicidad: El retiro se efectúa o no se realiza nada (si no hay fondos).

Consistencia: Nunca deja un saldo negativo, respetando reglas del sistema.

Aislamiento: Otros retiros simultáneos no afectan la ejecución individual.

Durabilidad: COMMIT asegura que el nuevo saldo quede guardado.

PROCEDURE TRANSFERIR(NUM_CUE_ORI, NUM_CUE_DES, VALOR)

```
SQL> CREATE TABLE TRANSFERENCIAS (  
2   NUM_TRA NUMBER PRIMARY KEY,  
3   FEC_HOR_TRA DATE NOT NULL,  
4   VALOR_TRA NUMBER NOT NULL,  
5   CUE_ORI VARCHAR(15) NOT NULL,  
6   CUE_DES VARCHAR(15) NOT NULL,  
7   FOREIGN KEY (CUE_ORI) REFERENCES CUENTAS(NUM_CUE),  
8   FOREIGN KEY (CUE_DES) REFERENCES CUENTAS(NUM_CUE)  
9 );
```

Tabla creada.

```
SQL> CREATE SEQUENCE seq_tra  
2   START WITH 1  
3   INCREMENT BY 1  
4   NOCACHE  
5   NOCYCLE;
```

Secuencia creada.

Creación del Procedure :

```
SQL> CREATE OR REPLACE PROCEDURE TRANSFERIR (  
2   P_NUM_CUE_ORI IN VARCHAR2,  
3   P_NUM_CUE_DES IN VARCHAR2,  
4   P_VALOR IN NUMBER  
5 ) AS  
6   V_SALDO_ORI NUMBER;  
7 BEGIN  
8   -- Verificar que la cuenta origen existe y tiene saldo suficiente  
9   SELECT SAL_CUE INTO V_SALDO_ORI  
10  FROM CUENTAS  
11  WHERE NUM_CUE = P_NUM_CUE_ORI;  
12  
13  IF V_SALDO_ORI < P_VALOR THEN  
14    RAISE_APPLICATION_ERROR(-20001, 'Fondos insuficientes.');15  END IF;  
16  
17  -- Debitar cuenta origen  
18  UPDATE CUENTAS  
19  SET SAL_CUE = SAL_CUE - P_VALOR  
20  WHERE NUM_CUE = P_NUM_CUE_ORI;  
21  
22  -- Acreditar cuenta destino  
23  UPDATE CUENTAS  
24  SET SAL_CUE = SAL_CUE + P_VALOR  
25  WHERE NUM_CUE = P_NUM_CUE_DES;  
26  
27  -- Verificar que la cuenta destino fue actualizada  
28  IF SQL%ROWCOUNT = 0 THEN  
29    RAISE_APPLICATION_ERROR(-20002, 'Cuenta destino no existe.');30  END IF;  
31  
32  -- Registrar transferencia con número autogenerated  
33  INSERT INTO TRANSFERENCIAS (  
34    NUM_TRA, FEC_HOR_TRA, VALOR_TRA, CUE_ORI, CUE_DES  
35  ) VALUES (  
36    seq_tra.NEXTVAL, SYSDATE, P_VALOR, P_NUM_CUE_ORI, P_NUM_CUE_DES  
37  );
```



Verificación del funcionamiento:

```
SQL> select * from transferencias;
```

ninguna fila seleccionada

```
SQL> select * from cuentas;
```

NUM_CUE	TIP_CUE	SAL_CUE	ANIO_APE	ID_CLI_PER
ACC1001	Ahorros	1700,5	2022	C001
ACC1002	Corriente	2300	2021	C002
ACC1003	Ahorros	900,75	2023	C003

```
SQL> execute TRANSFERIR('ACC1001', 'ACC1002', 200);
```

Procedimiento PL/SQL terminado correctamente.

```
SQL> select * from transferencias;
```

NUM_TRA	FEC_HOR_	VALOR_TRA	CUE_ORI	CUE_DES
1	30/06/25	200	ACC1001	ACC1002

```
SQL> select * from cuentas;
```

NUM_CUE	TIP_CUE	SAL_CUE	ANIO_APE	ID_CLI_PER
ACC1001	Ahorros	1500,5	2022	C001
ACC1002	Corriente	2500	2021	C002
ACC1003	Ahorros	900,75	2023	C003

Justificación:

Atomicidad: Si falla el débito, el crédito o la inserción en TRANSFERENCIAS, se revierte todo.

Consistencia: Se asegura que ambos saldos sean válidos, y se registra la transferencia.

Aislamiento: Transferencias simultáneas no se pisan entre sí gracias a ejecución transaccional.

Durabilidad: Al hacer COMMIT, los datos son permanentes incluso ante fallos del sistema.